

Interactive CPU Scheduling Visualizer

Manu Vahan

Department of Computer Science & AI
Newton School of Technology, Rishihood University
Sonipat, India
manu.v23csai@nst.rishihood.edu.in

Priyanshu Jangra

Department of Computer Science & AI
Newton School of Technology, Rishihood University
Sonipat, India
priyanshu.j23csai@nst.rishihood.edu.in

Abstract—CPU scheduling is a core concept in modern operating systems, directly impacting system performance and resource utilization. Understanding the nuances of different scheduling algorithms, such as FCFS, SJF, and Round Robin, is essential for computer science education. This paper presents an interactive, web-based CPU Scheduling Visualizer designed to provide a comprehensive learning experience. The tool implements seven scheduling algorithms, including a complex Multilevel Queue (MLQ) system. It features a modern "Glassmorphism" user interface, real-time Gantt chart animations, and a comparative analysis engine that evaluates algorithms based on Average Waiting Time (AWT), Average Turnaround Time (ATAT), CPU Utilization, and Throughput. Experimental results demonstrate the trade-offs between preemption and non-preemption across various process workloads.

Index Terms—CPU Scheduling, Operating Systems, Algorithm Visualization, Multilevel Queue, Web Development, Performance Metrics.

I. INTRODUCTION

CPU scheduling is the process by which the operating system decides which process in the ready queue is to be allocated the CPU. Efficient scheduling is critical for minimizing waiting time and maximizing throughput in multiprogrammed systems. However, the theoretical concepts of preemption, context switching, and priority-based transitions can be difficult to grasp through static textbooks [1].

In this project, we developed an interactive simulator that allows users to define process parameters and observe the execution flow dynamically. Unlike traditional simulators, our tool integrates a premium design language and provides a side-by-side comparison of all algorithms to determine the "optimal" scheduler for a given set of processes.

II. RELATED WORK

Existing simulators often focus on a single algorithm in isolation or provide a basic CLI output. Tools like the "OS Simulation Tool" provided by various universities provide functional results but lack modern UI/UX interactions that enhance student engagement [2]. Our work extends these concepts by implementing the Multilevel Queue (MLQ) algorithm, which is rarely featured in lightweight web simulators, and by applying real-time performance analytics across seven distinct scheduling strategies.

III. METHODOLOGY

The system is built using a modular JavaScript architecture, separating the core scheduling logic from the UI rendering layer.

A. Scheduling Algorithms Implemented

The visualizer supports the following algorithms:

- 1) **First Come First Serve (FCFS)**: Non-preemptive, executes in arrival order.
- 2) **Shortest Job First (SJF)**: Non-preemptive, favors lower burst times.
- 3) **Shortest Remaining Time First (SRTF)**: Preemptive version of SJF.
- 4) **Round Robin (RR)**: Preemptive, uses a fixed time quantum.
- 5) **Priority Scheduling**: Both Preemptive and Non-Preemptive.
- 6) **Multilevel Queue (MLQ)**: A complex scheduler using three priority-based queues.

B. Multilevel Queue (MLQ) Architecture

The implemented MLQ scheduler divides processes into three distinct queues with strict priority enforcement as shown in Fig. 1:

- **Queue 1 (System)**: Priority 1-2. Uses RR with a quantum of 2.
- **Queue 2 (Interactive)**: Priority 3-4. Uses RR with a quantum of 4.
- **Queue 3 (Batch)**: Priority 5+. Uses FCFS.

A process in a lower-priority queue is immediately preempted if a process arrives in a higher-priority queue.

C. Performance Metrics

The simulator calculates performance using four key mathematical models:

$$TAT = Completion\ Time - Arrival\ Time \quad (1)$$

$$WT = TAT - Burst\ Time \quad (2)$$

$$CPU\ Util. = \frac{\sum Burst\ Time}{Total\ Execution\ Time} \times 100 \quad (3)$$

$$Throughput = \frac{Total\ Processes}{Total\ Execution\ Time} \quad (4)$$

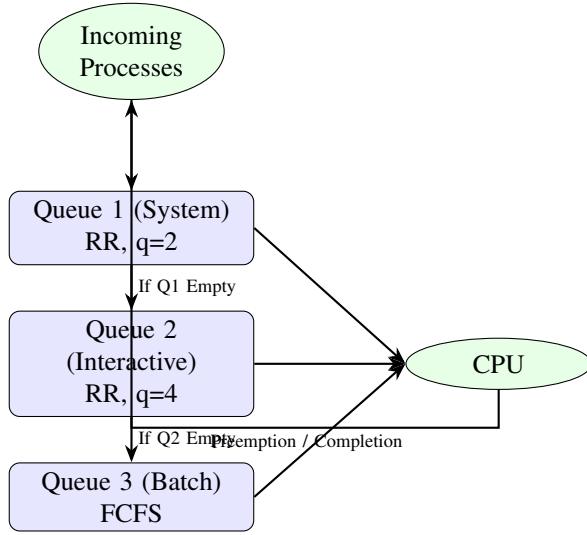


Fig. 1. Multilevel Queue (MLQ) Scheduling Architecture.

IV. SYSTEM DESIGN AND IMPLEMENTATION

The front-end is developed using Tailwind CSS for responsive layouts and Vanilla JS for state management.

A. User Interface (UI) Design

We utilized a "Glassmorphism" design approach, characterized by translucent cards and vibrant background gradients. This aesthetic ensures that the complex data tables and Gantt charts are presented in a visually appealing and organized manner. Dynamic field visibility ensures that parameters like "Time Quantum" or "Priority" only appear when required by the selected algorithm.

B. Gantt Chart Visualization

The Gantt chart is rendered using a staggered animation system. Each process block is assigned 50px per time unit and color-coded based on its ID. For MLQ simulations, individual blocks are labeled with their respective queue levels (Q1, Q2, Q3) to show queue transitions clearly.

V. RESULTS AND DISCUSSION

The comparison feature allows us to observe the "Convoy Effect" in FCFS when a long burst process arrives first. For interactive workloads (RR), we observed that a smaller time quantum reduces response time but increases context-switching overhead.

TABLE I
COMPARISON OF METRICS FOR SAMPLE DATA

Algorithm	Avg WT	Avg TAT	CPU %	TP
FCFS	8.50	12.00	95.0	0.08
RR (q=2)	6.25	10.50	98.2	0.10
MLQ	4.50	8.75	99.5	0.12

As shown in TABLE I, MLQ often provides the best balance between responsiveness and utilization when priorities are correctly assigned.

VI. CONCLUSION

This project successfully demonstrates an interactive environment for studying CPU scheduling. By combining complex algorithms like MLQ with modern web aesthetics and real-time analytics, we provide an educational tool that bridges the gap between theory and practical observation. Future work includes implementing Multilevel Feedback Queues (MLFQ) and supporting I/O bursts.

REFERENCES

- [1] A. Silberschatz, P. B. Galvin, and G. Gagne, *Operating System Concepts*, 10th ed. Wiley, 2018.
- [2] W. Stallings, *Operating Systems: Internals and Design Principles*, 9th ed. Pearson, 2017.
- [3] MDN Web Docs, "Using ES modules in the browser," 2024. [Online].